Vdcap.AI

4-Style Synchronized AI Video Captioning Platform

Track: Video-Captioning Pipeline

Built for: AMD Developer Hackathon: ACT II

Developed by: Rishabharaj Sharma

The Core Problem & Vdcap.AI Solution

The Problem

- Single-Style Limitations: Current caption generators only support a single, generic style, forcing creators to manually adapt transcripts for different audiences.

- High Resource Costs: Manually styling subtitles across multiple tones (casual, tech, professional) requires extensive video editing and caption re-writing.

- Engagement & Accessibility Gaps: Static or poorly-targeted captions reduce viewer engagement, failing to hook audiences on social media platforms.

Our Solution: Vdcap.AI

- Automatic 4-Style Generation: Transcribes video and uses LLMs to concurrently generate Formal, Sarcastic, Tech-Humor, and Casual-Humor captions.

- Synced Multi-Player Comparison: Implements a 4-quadrant layout where all 4 streams play in frame-perfect synchronization, allowing instant subtitle evaluation.

- Low-Overhead Pipeline: Automates audio extraction, speech transcription, vision analysis, and drawtext overlays in a single streamlined execution loop.

Technical Architecture & Processing Pipeline

End-to-End Pipeline Execution in the Codebase:

*** Step 1: Input Ingestion & Verification:**
FastAPI backend accepts video files (MP4, WebM, etc.). Uses FFprobe to programmatically validate duration, format, and check for an audio track.

*** Step 2: Dual-Mode Extraction Pipeline:**
If audio is present: FFmpeg extracts a 16kHz mono WAV file for transcription. If video is mute: OpenCV splits keyframes at 1 FPS to extract visual representative context for captioning.

*** Step 3: Speech-To-Text Transcription:**
Whisper processes the extracted WAV file to generate high-accuracy transcripts complete with word-level timestamps.

*** Step 4: Fireworks AI LLM Captioning Engine:**
Dispatches transcript and visual frames to Fireworks AI using the GLM-5.2 (glm-5p2) model. Four custom-engineered system prompts guide the model to simultaneously generate Formal, Sarcastic, Humorous-Tech, and Humorous-NonTech subtitles.

Synchronized Playback & Subtitle Burn-In

1. Synchronized 4-Quadrant Player

- Native synchronized playback built using HTML5 video nodes and Vanilla JavaScript.

- A centralized event controller captures play, pause, seek, and ratechange events on the primary player and mirrors them instantly to the other three quadrants.

- Color-coded design maps to each specific caption style:
  * Formal: Blue Theme
  * Sarcastic: Pink Theme
  * Humorous-Tech: Purple Theme
  * Humorous-NonTech: Emerald Theme

2. Subtitle Burn-In & Rich Exports

- Subtitles are permanently overlayed onto separate video copies via FFmpeg's drawtext filter.

- Custom font configurations (Inter-Bold), outline strokes, sizes, and padding are applied dynamically to render high-contrast subtitle blocks.

- Robust export service packages results in various formats:
  * SRT files matching each style's timestamps
  * Formatted JSON files for programmatic reuse
  * Captioned MP4 downloads
  * Combined ZIP containing files and HTML report summary

Model Fine-Tuning & Future Roadmap

Fine-Tuning Pipeline (LoRA)

- Codebase includes a fine-tuning module (train_captioner.py) for domain adaptation.

- Configured to train on MSR-VTT (Video-to-Text) and ActivityNet Captions datasets.

- Integrates Low-Rank Adaptation (LoRA) to adapt LLM layers with reduced GPU memory footprint.

- Standard hyperparameters (rank=16, alpha=32, batch_size=8) allow rapid training on AMD ROCm compute instances.

Project Future Scope

- Multi-Language Support: Broaden caption translation into multiple global languages (French, Spanish, Hindi).

- Live Webcam/Stream Integration: Implement real-time audio chunk processing and streaming caption burn-in.

- Enterprise Dashboard: Add user databases to save generation histories and track API consumption rates.

- Mobile Companion App: Develop React Native companion app for mobile content creators.